Recap Lezione 3

Rif, sempre: https://github.com/feroda/its-jesi-lessons/

Fase 1 - Correzione esercizi svolti nella lez. 2

TODO: mettere soluzioni

Fase 2 - Rivisitazione architettura software del troubleshooting lez. 1

Nella prima lezione è stata presentata un'architettura software con un server, un sistema operativo e 2 servizi in ascolto (gestiti da Docker). Il prof. ha riproposto le stesse slides con una metafora del centro commerciale per spiegare le basi dello stack TCP/IP, in particolare i livelli 3, 4, 7 e l'incapsulamento con 2 buste riferito a questi 3 livelli (3 e 4 le buste esterne, 7 il messaggio)

Con essi sono stati associati gli strumenti di test e debug lato client (ping - livello 3, nc/telnet - livello 4, curl - livello 7 per HTTP) e lato server (ss/netstat per i servizi attivi e le porte su cui ascoltano, journalctl per i log, tcpdump per analisi dettagliate su tutto il pacchetto)

Rif: https://github.com/feroda/its-jesi-lessons/blob/main/logging/slides/Troubleshooting.pdf

Fase 3 - Fondamenti di Docker

Come si è accennato nell'architettura della prima lezione c'erano 2 servizi eseguiti in containers dentro il sistema di containerizzazione Docker, per questo si è spiegato questa tecnologia in modo da poter provare poi la soluzione di logging e monitoraggio autonomamente.

Docker gestisce container. Un container può essere considerato una *macchina virtuale leggera* (non è esatto, ma vediamo le differenze):

- Una macchina virtuale gira su hardware virtualizzato e su un kernel dentro questo hardware virtualizzato e fa eseguire un intero sistema operativo
- Un container gira su un unico kernel che è in contatto con la macchina fisica, è gestito da un servizio (nel nostro caso **dockerd**) ed è fatto per eseguire UNA sola applicazione. Più container insieme rappresentano più applicazioni e gestirli si dice "orchestrare i container"

Abbiamo presentato i concetti di "motore", ossia il demone **dockerd** che riceve i comandi dal client (il comando) **docker** ed abbiamo provato il comando `docker run hello-world` . Il demone dockerd:

1. tenta di trovare un'immagine locale chiamata `hello-world`
2. se non la trova la scarica dal **registry**, ossia il catalogo delle immagini dei container (di solito hub.docker.com)
3. una volta scaricata l'immagine, esegue il container
4. mostra l'output che sarebbero i log e sono visibili con **docker logs**

i container attivi si possono vedere con `docker ps` e se si vogliono vedere anche quelli non attivi (tra cui il container di hello-world) si deve usare `docker ps -a` .

Rif: https://github.com/feroda/its-jesi-lessons/blob/main/logging/slides/Docker.pdf fino alla slide 9